



Projektni zadatak

Automatizacija poslovnih procesa: Studija slucaja automatizacije slanja predispitnih obaveza

Predmet: IT380 Inzenjerstvo i integracija sistema

Tip dokumenta: Projektna Dokumentacija

Datum: 1.3.2025

Student: Veljko Rakic

Index: 5813

Asistent: Milos Milasinovic

Table of Contents

1. Uvod.....	3
2. Teoretska postavka	4
2.1 Poslovni procesi.....	4
2.1.1 Sta su poslovni procesi?	4
2.1.2 Vrste poslovnih procesa i njihovi elementi	5
2.1.3 Modelovanje (BPMN)	6
2.2 Automatizacija poslovnih procesa	9
2.2.1 Sta znaci automatizacija poslovnih procesa?	9
2.2.2 Ciljevi i koristi automatizacije	9
2.2.3 Uspesnost automatizovanog procesa.....	10
2.2.4 Koraci u automatizaciji digitalnog poslovnog procesa	11
3. Studija slucaja: Automatizacija poslovnog procesa slanja predispitnih obaveza.....	12
3.1 Definisanje poslovnog procesa.....	12
3.2 BPMN Model	13
3.3 MuleSoft Anypoint Studio i MuleSoft.....	14
3.4 DEMO.....	15
3.4 Tehnicke specifikacije aplikacije za automatizaciju	16
4. Zakljucak	31
5. Literatura	31

1. Uvod

Za ovaj projektni zadatak, iz predmeta inženjerstva i integracija sistema, odlucio sam se za temu *Automatizacija Poslovnih Proces*.

Takodje u ovom projektu i pod ovom temom ce biti odradjen i studija slucaja automatizacija slanja fajlova i dokumenta, domacih koje imamo na nasem fakultetu i koje saljemo nasim profesorima/asistentima. Ovde ce upravo to slanje predstavljati poslovni proces koji treba biti automatizovan.

Ovu temu sam izabrao zato sto mi je automatizacija u tehnologiji jako zanimljiva, a uz ovaj projektni zadatak mogu je istraziti i u drugom polju, specificno u polju poslovnih procesa kao i to samo polje. Konkretno cemo sa ovom studijom slucaja resiti problem stalnog pisanja i slanja mejlova koji nakon izrade znaju biti zamorni i dosadni. Gde oni cak i podlezu greskama neuspelog ili pogresnog slanja. Resenjem ovog problema stedimo vreme i napor kroz koji studenti kolektivno trose na pisanje i sastavljanje mejlova pri slanju svojih predispitnih obaveza, koji veoma lako mogu biti zamenjeni automatizovanim i genericnim slanjem.

Takodje smatram da ce ovaj primer biti dobar prikaz objedinjenog znanja sa ovog predmeta a posebno po alatu i tehnikama koje se koristi pri izradi spomenutog.

U teoretskoj postavci mi cemo objasniti sve bitne teorijske okvire koje moramo razumeti pre nego sto prodjemo na samu studiju slucaja. Takodje cemo opisati i tehnologije i tehnike koji se koriste i implementiraju u realnom svetu nad realnim sistemima, i kako mozemo to koristiti u nasem primeru. Kada prodjemo kroz teoretski okvir, preci cemo na sam primeri i kako je on odradjen i da li ispunjava nase postavljene zahteve. Kroz screenshot-ove, dijagrame pa cak i delove koda cemo detaljno prikazati nas primer, a kroz studije takodje cemo se truditi da sto detaljnije i preciznije opisujemo sta i kako smo radili.

2. Teoretska postavka

2.1 Poslovni procesi

2.1.1 Sta su poslovni procesi?

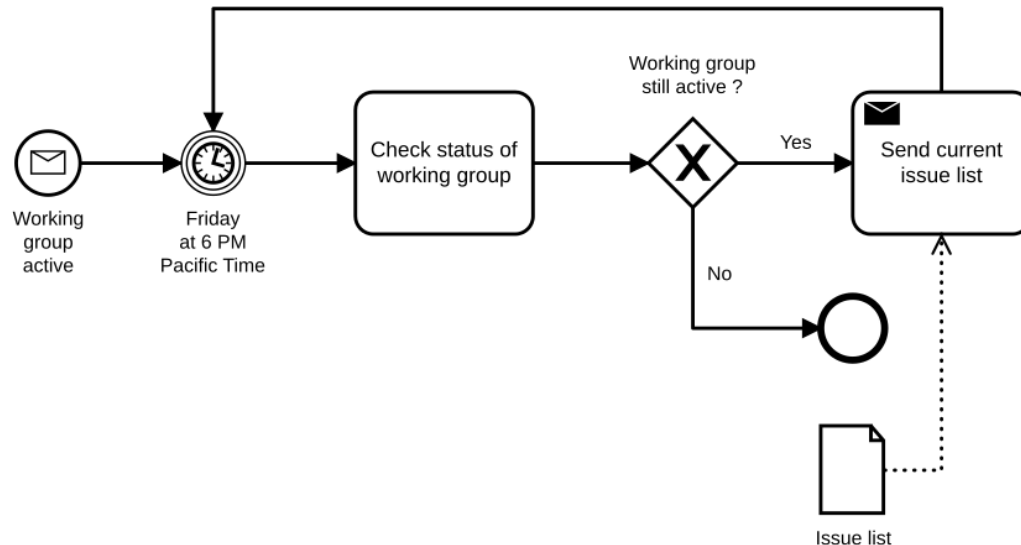
Da bi uopšte dosli na temu kako se baviti automatizacijom poslovnih procesa, prvo moramo da znamo sta je poslovni proces.

Poslovni proces jeste skup povezanih aktivnosti ili operacija koje se izvršavaju od strane pojedinaca ili tehnologije, u datom redosledu, u organizaciji kako bi se uspešno postigao željeni cilj ili produkt. Ovo predstavlja efikasan i način za ostvarivanje poslovnih ciljeva i implementira se u svakoj organizaciji i sferi rada, naravno u različitim se i drugacije implementira ali je koncept isti.

Takodje treba spomenuti i karakteristike poslovnog procesa a to je da je poslovni proces: ponovljiv, proces je fleksibilan i nije krut, ima jasno definisak pocetak i kraj, proces je takodje merljiv.

Poslovni procesi mogu biti interni u sklopu organizacije i mogu biti eksterni gde se javlja ukljucivanje rada sa klijentima u sam proces, ali sam poslovni proces ne mora biti vidljiv klijentu.

Same aktivnosti kojima se procesi bave mogu biti složene ili jednostavne, gde mogu gadjati bilu koju oblast unutar jedna organizacije npr. oblast prodaje, oblast marketinga, oblast tehnologije... Takodje poslovni procesi se obicno vizualizuju pomocu dijagrama toka aktivnosti sa razlicitim tackama odlucivanja koje obuhvataju sve clanove tima u tom poslovnom procesu, ovo dosta olaksava upravljanje i razumevanje kako treba da ide sam poslovni proces i koji cilj treba postici.



Slika 1: Primer dijagrama toka u jednom poslovnom procesu.

Svaki poslovni proces pocinje sa misijom koja ima kranji cilj, koja je obicno postavljena zbog spoljnog dogadjaja, i završava se postizanjem postavljenog cilja, gde se kao rezultat dobija neka vrednost ili za organizaciju ili za klijenta. Poslovni procesi mogu imati i podpreocese, kao i vlasnika procesa i odgovornih za neometano odvijanje procesa.

2.1.2 Vrste poslovnih procesa i njihovi elementi

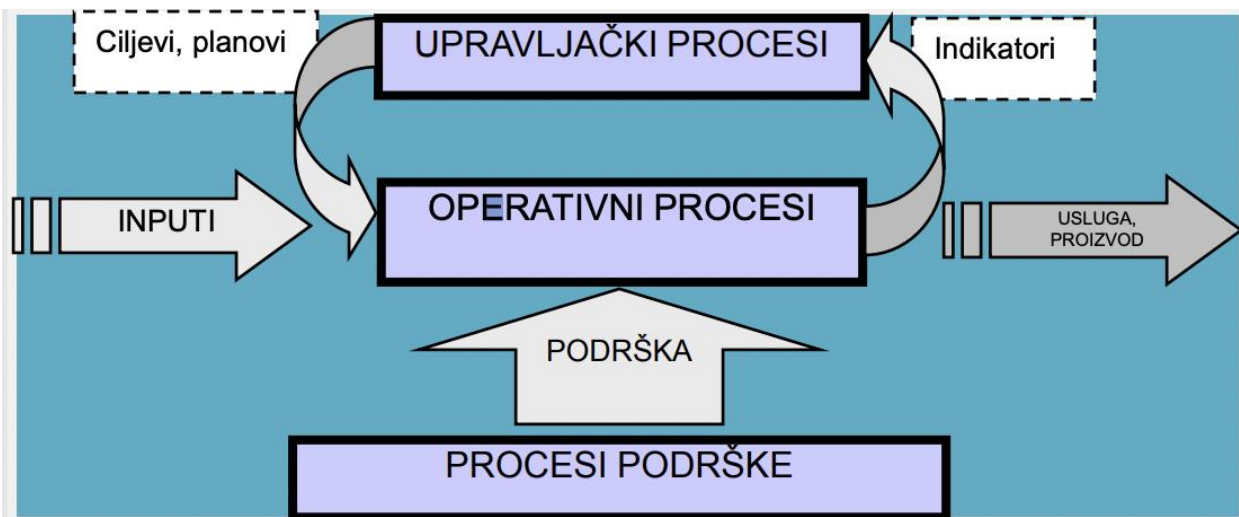
U svakoj organizaciji postoje poslovni procesi i oni se mogu podeliti na tri kategorije, a to su operativni ili osnovni procesi, upravljački procesi i podržavajući procesi.

Operativni procesi jesu osnova svake organizacije jer oni direktno stvaraju vrednost za klijente i korisnike. Aktivnosti koje su uključene u ovaj proces su usmerene na proizvodnju robe, pružanju usluga ili isporuku proizvoda samom klijentu. Najbitniji deo u ovom procesu jeste dodana vrednost. Ovo znaci da svaki korak u ovom procesu treba da doprinese kranjnjem proizvodi ili usluzi. Ovaj tip procesa ima najveći prihod za organizaciju i zato je osnovni proces. Organizacije ili kompanije koje uspeju da optimizuju i povecaju efikasnost svojih operativnih procesa, smanjuju troškove i samim tim poboljšavaju kvalitet proizvoda ili usluga, sto im daje korak ispred svoje konkurencije.

Da bi operativni procesi uopste imali direkciju, na snagu dolazi sledeca vrsta procesa, a to je upravljački proces. Oni se bave svim procesima koji osiguravaju da organizacija radi u skladu sa svojim ciljevima. U proces to uključuje donosenje ključnih odluka, planiranja, pracenja i analiza kao i upravljanje resursima i tokova drugih procesa. Ovo se sve može objediniti pod pojmom strateskog planiranja gde se postavljaju dugorocni ciljevi i smernice ka njihovom ostvarivanju. Sve odluke koje se donose trebaju proci prvo kroz detaljnu analizu menadzerskog odeljenja ili ako kompanija sadrzi odeljenje posebno za tu delatnost.

Na kraju imamo i podržavajuće procese koji ne utice direktno na stvaranju vrednosti za kranje korisnike. ali igraju posebnu ulogu u omogucavanju nesmesanog rada za operativne i upravljacke procese. Ovo uključuje sve uloge podrške kao sto su IT podrška, upravljanje ljudskim resursima, pravna i administrativna usluga, logistika, nabavka...

Iako su ovi različiti u svojim funkcijama oni su blisko povezani i za uspesan rad jedne organizacije oni moraju raditi medjusobno u balansu.



Slika 2: Dijagram koji pokazuje usku povezanost i rad razlicitih procesa u organizaciji.

Ne smemo zaboraviti ni klucne elemente poslovnih procesa, koji predstavljaju komponente koji omogucavaju procesima da funkcionišu na strukturisan nacin. Kao i na gore prikazanoj slici, svaki poslovni proces se sastoji od ulaznih podataka, aktivnosti, resursa, izlaznih podataka.

Ulazni podaci predstavljaju pocetne informacije, materijale, resurse ili zahteve koji su potrebni da bi proces poceo da se izvrsava. Oni mogu biti razlicitih izvora.

Aktivnosti jesu niz medjusobno povezanih koraka koji se sprovode kako bi se ulazni podaci transformisali u ciljani rezultat. One mogu biti manuelne ili automatizovane. One se dokumentuju kroz procesne dijagrame radi lakseg pregleda i optimizacije.

Resursi su sve što je potrebno za sprovođenje aktivnosti u okviru poslovnog procesa, a u njih spadaju ljudski resursi, finansijski, tehnološki, softverski kao i resursi infrastrukture. Iako bi imali dobro određene procese bez resursa oni ne vrede ništa.

Na kraju imamo izlazne podatke što nam predstavlja rezultat našeg procesa u vidu proizvoda ili usluge. Cilj svakog procesa bilo koje vrste je da obezbedi izlazne podatke koji su vredne ili za klijente ili za organizaciju.

2.1.3 Modelovanje (BPMN)

Sada kada znamo kakvi sve tipovi poslovnih procesa postoje i kakvi elementi su potrebni za njihov uspešan rad, trebamo precizirati preko oblasti koja se bavi modelovanjem i grafičkim prikazom samih tih procesa. Glavni način modelovanja poslovnih procesa jeste preko Business Process Model and Notation ili BPMN.

BPMN jeste grafička reprezentacija za specifikaciju poslovnih procesa. On jeste i standard za modeliranje poslovnih procesa zasnovan na tehnici dijagrama toka koji možemo reći da je sličan dijagramima aktivnosti iz UML-a. Sam njegov cilj jeste da podrži upravljanje poslovnim procesima za tehničke korisnike kao i za poslovne korisnike.

Specifično on je dizajniran da obezbedi standardnu notaciju koja je lako razumljiva svim poslovnim akterima, poslovnim analitičarima, tehničkim programerima i poslovnim menadžerima. On usvajanjem zajedničkog jezika za opisivanje procesa pomaže da se izbegnu bilo kakve nejasnoće u komunikaciji koje nastaju pri dizajniranju samog procesa. BPMN je samo jedan od modela u ovom polju postoje i mnogi drugi.

Sada ćemo precizirati na neke opšte specifikacije BPMN-a.

Što se tiče obima, BPMN je ograničen da podržava samo koncepte modeliranja primenljivih na poslovne procese, dok druge vrste modeliranja su van njegovog opsega. Neke koje BPMN ne podržava su organizacione strukture, funkcionalni kvarovi kao i modeli podataka. On takođe nije dijagram toka podataka iako pokazuje tok poruka i artefakata podataka sa aktivnostima.

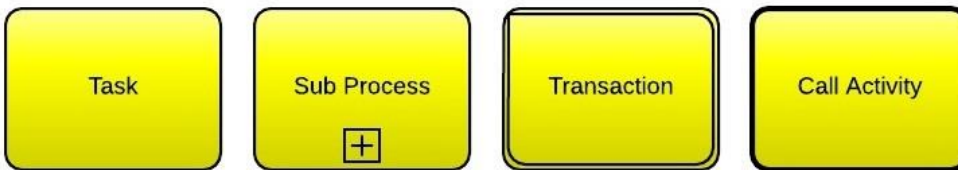
BPMN modeli su izraženi jednostavnim dijagramima konstruisani od skupa grafičkih elemenata, koji za poslovne korisnike i programere pojednostavljuju razumevanje toka procesa i aktivnosti. On ima četiri osnovne kategorije elemenata što su flow objects, connecting objects, swim lanes i artifacts. U flow objects spadaju events, activities, gateways. U connecting objects spadaju sequence flow, message flow, association. Dok u swim lanes spadaju pool, lane, Dark Pool. I na kraju u artifacts data objects, group i annotation.

Event predstavlja nešto što se događa za razliku od aktivnosti koja se izvršava. Obeležava se krugom a ikone unutar njega označavaju tip događaja. Event može biti ili catching koji hvata dolazni događaj ili throwing koji šalje događaj. Start Event pokreće proces, ima tanak okvir i uvek je catching. Intermediate Event se događa između početka i kraja procesa, ima dupli okvir i može biti ili catching ili throwing. End Event označava kraj procesa, ima deobeo okvir i uvek je throwing.



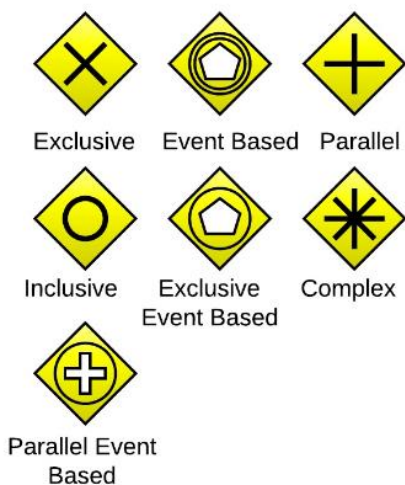
Slika 3: Grafički prikaz event objekata.

Activity predstavlja posao koji treba da se obavi, on je prikazan pravougaonikom sa zaobljenim uglovima. On može biti atomic što znači da se ne deli dalje ili može biti compound. Sama reč activity je generična i predstavlja više terimn za rad koja ta specifična organizacija obavlja. Task je najmanja jedinica rada i ne može se dalje razložiti i predstavlja atomic acitivity. Sub-process sakriva ili otkriva dodatne detalje procesa, on takođe ima spostvene start i end događaje. Transaction je poseban sub-process gde sve aktivnosti moraju biti završene zajedno ako jedne ne uspe sve se ponistavaju. Call Activity ponovo koriscenje globalnog procesa.



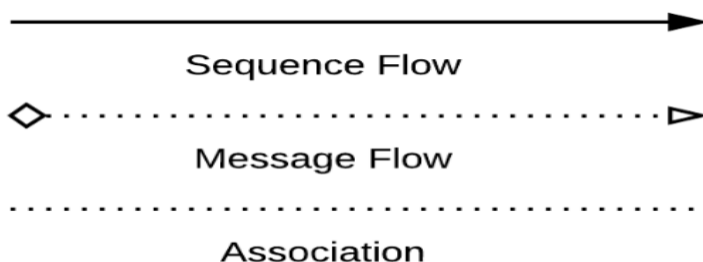
Slika 4: Graficki prikaz activity objekata.

Gateway je predstavljen dijamantskim oblikom i određuje grananje i spajanje tokova na osnovu uslova. Ti uslovi mogu biti razni gde se negde kreiraju alternativni tokovi ali samo jedan može biti izabran, može biti event based gde zavisi od događaja, parallel kreira paralelne tokove, inclusive kreira alternativne tokove ali se svi ocenjuju, complex je složeno sinhronizacijsko ponašanje, exclusive event based kao exlusive samo sa event-om, parallel event based pokrece dva paralelna na osnovu događaja.



Slika 5: Graficki prikaz gateway objekata.

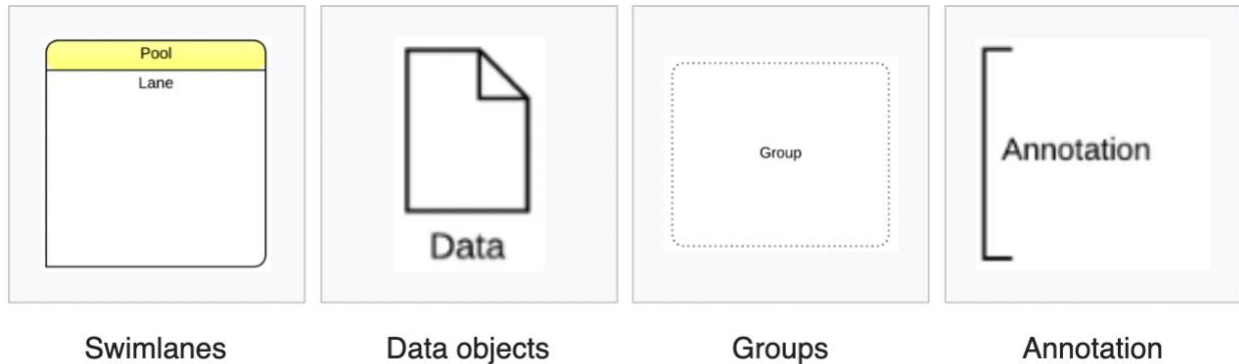
Svi ovi flow objects su povezani sa connecting objects koji ima tri tipa. Sequence Flow koji pokazuje da se aktivnosti izvrsavaju ekvencijalno kako je strelica povucena. Message Flow je isprekidana linija sa otvorenim krugom i strelicom i prikazuje razmenu poruka izmedju organizacionih jedinica. Association je tackasa linija i povezuje artefakte ili tekst sa flow objektima, može imati i strelicu da oznaci smer veze.



Slika 6: Graficki prikaz connecting objekata.

Swim lanes vizuelno organizuju aktivnosti i sastoje se od pool i lane. Pool predstavlja glavne ucesnike procesa i moze biti otvoren ili sazet, dok lane kategorizuje aktivnosti unutar pool-a prema funkciji ili ulozi koju igra.

Artifacts dodaju dodatne informacije i ukljucuju data objects koji prikazuju potrebne ili proizvedene podatke. Group obelezava povezane aktivnosti ali ne utice na tok dijagrama dok annotation dodaje objasnjenje za bolje razumevanje nekog dela dijagrama.



Slika 7: Graficki prikaz swimlanes, data objects, groups i annotation.

Ovaj opis i semantika BPMN-a je jako bitna jer se primenjuje u svakoj organizaciji i pri svakom poslovnom procesu, a kasnije ce nam trebati kada budemo razvijali poslovni proces u nasoj studiji slucaja.

2.2 Automatizacija poslovnih procesa

2.2.1 Sta znaci automatizacija poslovnih procesa?

Ukratko receno automatizacija poslovnih procesa, Business Process Automation ili BPA, znaci da organizacija primenjuje tehnologiju u cilju optimizacije i automatizacije svojih poslovnih procesa sto predstavljaju, kao sto smo gore naveli, sve zadatke i aktivnosti u toj organizaciji.

Ovo predstavlja nacin da se kroz tehnologiju i automatizaciju pri izradi zadatka smanje ljudski naponi i uštedi vreme. Po definiciji sama automatizacija poslovnih procesa, u odnosu na automatizaciju sa tehnologijom, se odnosi na koriscenje softverskih resenja, skripti i integracionih platformi za upravljanje i izvršavanje ponavljajucih zadataka bez manuelne ljudske intervencije.

Proces automatizacije obuhvata identifikaciju kljucnih zadataka ili aktivnosti u toku poslovnih procesa koji su vremenski zahtevni ili skloni greskama i pokusaj njihove zamene automatizovanim sistemom.

Ovo ne znaci da neophodno se najvise automatizuju najzahtevniji vremenski i napodlozniji greskama zadaci, vec nasuprot u realnim situacijama, mali zadaci koji nisu toliko zahtevni ali idalje zahtevaju vreme i moze se na njima pogresiti su najpogodniji za automatizaciju, kako bi zaposleni mogli da se bave vise zahtevnim zadacima.

U ovom radu takodje specificno pricamo o automatizaciji digitalnih zadataka, dok postoje automatizacije kao sto su automatizacija radnih tokova, robotizovana automatizacija procesa, kongnitivna automatizacija i integraciona automatizacija.

2.2.2 Ciljevi i koristi automatizacije

Kao i svemu sto se primenjuje u biznisu i njenim organizacijama i kompanijama, tako i automatizaciji jeste cilj povecanju doprinosa i smanjivanju troskova. Automatizacija ovo radi indirektno, smanjivajuci obaveze i vreme u određenim delovima poslovnih procesa, ako ne i samog celog poslovnog procesa.

Smanjenje manualnog rada i gresaka jeste jedan od glavnijih ciljeva. Ovo ukljucuje eliminaciju ponavljajucih zadataka koji su skloni greskama kao i povecavanje tacnosti u obradi podataka, sto je na danasnjem trzistu veoma bitno sa obzirom da je veliki broj kompanija i organizacija visoko digitalizovano.

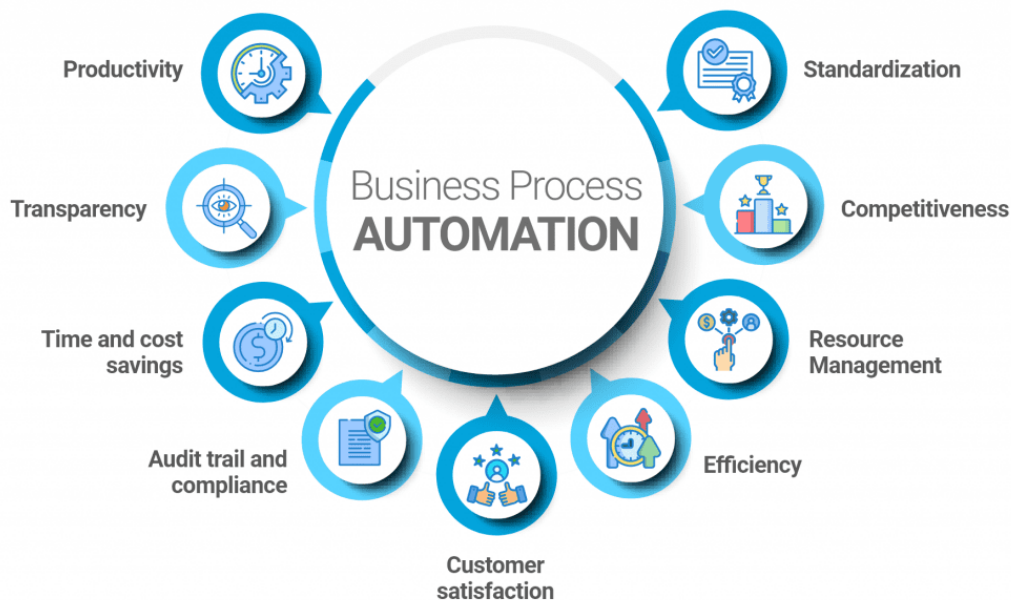
Takodje jedan od bitnih ciljeva same organizacije je povecavanje efikasnosti i produktivnosti svojih zaposlenih, a ovo postizu ubrzavanjem procesa kroz automatizaciju radnih tokova, sto indirektno omogucava zaposlenima da se fokusiraju na strateske zadatke umesto rutinskih.

Automatizovani sistemi rade na osnovu definisanih pravila i algoritama sto znaci da procesi postaju precizniji i tacni u odnosu na varijaciju koja bi nastala da je obavljeno ljudskom rukom. Ovo pospesuje standardizaciju procesa.

Samim tim sto je brza obrada zahteva i odgovra klijenata, to se poboljsava i njihovo korisnicko iskustvo.

Ovim sistemom se i unapredjuje bolja kontrola nadzora nad procesima jer automatizovani sistemi omogucavaju bolji uvid u kljucne pokazatelje performansi ili KPIs, gde se takodje postize i bolja transparentnost i laska analiza.

A najbitnije za kompanije, ovo sve smanjuje operativnih troskova jer je smanjena potreba za rucnim unosom podataka i administracijom. Dok optimizacija resursa omogucava bolju raspodelu budzeta i smanjuje nepotrebne troskove.



Slika 8: Ciljevi BPA.

Ukratko ciljevi automatizacije poslovnih procesa su usmereni na povećanje produktivnosti, smanjenje gresaka i unapređivanje operativne efikasnosti kroz optimizaciju resursa i bolju analitiku, organizacije mogu postići dugoročne koristi, biti konkurentnije i unaprediti kvalitet svojih usluga.

2.2.3 Uspesnost automatizovanog procesa

Nakon što se proces automatizuje potrebno je i proceniti koja je efikasnost i pouzdanost samog tog procesa. Ovo merenje izvršavamo sa ključnim indikatorima učinka ili KPI (Key Performance Indicators). KPI predstavlja sve merljive vrednosti koje pokazuju koliko je automatizacija doprinela poboljšanju poslovanja.

Efikasnost i brzina procesa je jedna od osnovnih metrika i ciljeva automatizacije. U KPI to uključuje Cycle Time ili prosečno vreme izvršenja procesa, gde se meri koliko vremena je potrebno da se završi određeni proces od početka do kraja. Ako je upoređeno i vidi se da je manje vremena potrebno sa automatizacijom, znaci da je ona efikasna. Response time meri koliko brzo automatizovan sistem odgovara na zahteve, dok analiza procenta povećanje produktivnosti nam direktno pokazuje koliko je vremena uštedjeno.

Kvaliteti i tačnost je metrika koja bi trebalo da da u uvid smanjeni broj gresaka i povećanu preciznost aktivnosti. Error Rate ili stopa gresaka meri procenat neispravnih ili netacnih izvršenja procesa. Niza stopa gresaka uazuje na bolji kvalitet automatizacije. Dok compliance rate procenjuje u kojoj meri procesi ispunjavaju interne standarde. Na kraju ova metrika meri i koliki je broj potrebnih rucnih intervencija zbog gresaka ili neocekivanih problema u automatizaciji.

Return on Investment ili ROI je za organizacije jedan od ključnih razloga za uvođenje automatizacije zbog smanjivanja operativnih troškova i povećanja profitabilnosti. Ovde se meri trosak obavljanja procesa gde se gleda koštanje izvršenja jednog procesa pre ili posle automatizacije. ROI meri koliko se investicija u automatizaciju isplatila u odnosu na ostvarene uštede i povećanu produktivnost. Takođe se meri i smanjenje broja radnih sati.

Korisnicko iskustvo se meri sa CSAT ili Customer Satisfaction Score, tacnije meri se koliko su korisnici zadovoljni brzinom i tacnoscu automatizovanih usluga. Meri se i nivo korisnicke interakcije kao i vreme odgovora korisnicke podrške.

Na kraju se mere i faktori kao sto su sposnostonost da se prilagodi rastu i promenama u poslovanju. Stopa skalabilnosti meri koliko se lako automatizovani proces moze prosiristi na veci obim posla ili druge delove firme. Isto se procenjuje koliko je lako napraviti izmene u automatizovanom procesu bez znacajnih troskova ili problema.

2.2.4 Koraci u automatizaciji digitalnog poslovnog procesa

Sada kada smo upoznati sa osnovnim defimicijama i teorijama o poslovnim procesima i njegovim automatizacijama, prikazacemo koji su koraci pri automatizacij jednog digitalnog poslovnog procesa, a kasnije u studiji slucaja ih i implementirati.

Prvi korak jeste identifikacija procesa koji treba automatizovati i njegova analiza. Znaci pre nego sto mozemo nesto automatizovati mi se moramo upoznati sa njim i analizirati da li je on pogodan za automatizaciju.

Neka od pitanja koja se trebaju odgoviti u ovom delu su, koji poslovni procesi se ponavljaju i zahtevaju nepotreban manuelni rad? Gde su uska grla, greske i trosak vremena? Koji su ciljevi automatizacije u smislu troskova, efikasnosti, korisnickog iskustva?

Ovakva analiza se uglavnom vrsi pomocu pomenutog BPMN-a.

Drugi korak nakon sto se identifikuje proces koji treba da se automatizuje, jeste precizno definisanje ciljeva i ocekivanih rezultata automatizacije. Ovo se resava pitanjima kao sto su, koji su ulazi i izlazni podaci procesa? Koji se tacno delovi procesa mogu automatizovati a gde je tacno potrebna ljudska intervencija?

Treci korak nakon postavljenih ciljeva jeste izbor odgovarajuce tehnologije koja ce podrzati ovu automatizaciju. Ovde ima dosta razlike u zavisnosti od samog procesa i postojeceg tehnoloskog okruzenja. Neki od najcesce koriscenih alata su RPA ili Robotic Process Automation sto su softverski botovi za automatizaciju ponavljajucih zadataka kao sto su UiPath ili Blue Prism. Sami softveri za slozene radne tokove poput Camunda ili IBM BPM. Integracioni alati kao sto su MuleSoft Anypoint Studio ili Apache Camel.

Cetvrti korak jeste dizajniranje i modelovanje automatizovanog procesa. Definisanje radnih tokova, integracija postojećih sistema, donosenje odluka, obrada izuzetaka su jedna od pravila koja trebaju da se postuju pri ovom koraku.

Peti korak jeste implementacija automatizacije gde kada je model završen, prelazi se na tehnicku implementaciju. To ukljucuje razvoj i konfiguraciju softverskih resenja ili integracija. Definisanje API konektora i pravila poslovne logike kao i podesavanje sistema za nadzor i pracenje tog procesa.

Sesti korak jeste testiranje pre nego sto se ode u produkciju i ima 3 vrste testiranja, funkcionalno, integraciono, testiranje performansi. Funkcionalno je da li svi koraci rade kako je planirano, integraciono je da li su svi sistemi pravilno povezani, a testiranje performansi služi da se vidi da li sistem moze podneti veliki broj zahteva.

Sedmi korak obuhvata implementaciju gde se vrsi migracija na produkcionu infrastrukturu, obuku zaposlenih za rad na novim sistemom i definisanja plana podrške.

Na kraju u osmom koraku imamo pracenje automatizovanog procesa i njegova optimizacija.

3. Studija slucaja: Automatizacija poslovnog procesa slanja predispitnih obaveza

3.1 Definisanje poslovnog procesa

Nakon sto smo u teoretskoj postavci objasnili sta su poslovni procesi i kako se vrši njihova automatizacija, sada cemo to primeniti na realnoj studiji slucaja, sa primerom poslovnog procesa slanja predispitnih obaveza.

Za primer sam uzeo rad organizacije naseg univerziteta Metropolitana, specificno rada Fakulteta Informacionih Tehnologija, gde cu u studiji slucaja sa organizacijom oslovljavati i misliti na rad FIT-a.

U ovoj organizaciji, tokom rada izmedju studenata i profesora/asistenata, javlja se poseban cesto ponovljiv poslovni proces koji oduzima vreme studentima koji je takodje obavezan, a taj proces je slanje svojih predispitnih obaveza putem email-a svojim asistentima. Ovaj proces se ponavlja svaki put kada student završi jednu od svojih predispitnih obaveza iz svakog predmeta.

Ovaj proces se sastoji od aktivnosti gde on prvo sastavlja dokument sa potrebnim fajlovima, naziva ga po konvenciji imenovanja predispitnih dokumenata u okviru organizacije, zipuje fajl, otvara browser i ulazi na mail organizacije, kreira novi mejl gde imenuje primaoca, pise naslov i subjekat, kao i telo sa porukom i dodatim fajlom koji sadrži predispitnu obavezu, gde nakon svih ovih koraka se taj mejl salje na ocenjivanje asistentu.

Sam proces je eksterni tip procesa, jer ovde student predstavlja klijenta, dok profesori/asistenti pružaju usluge.

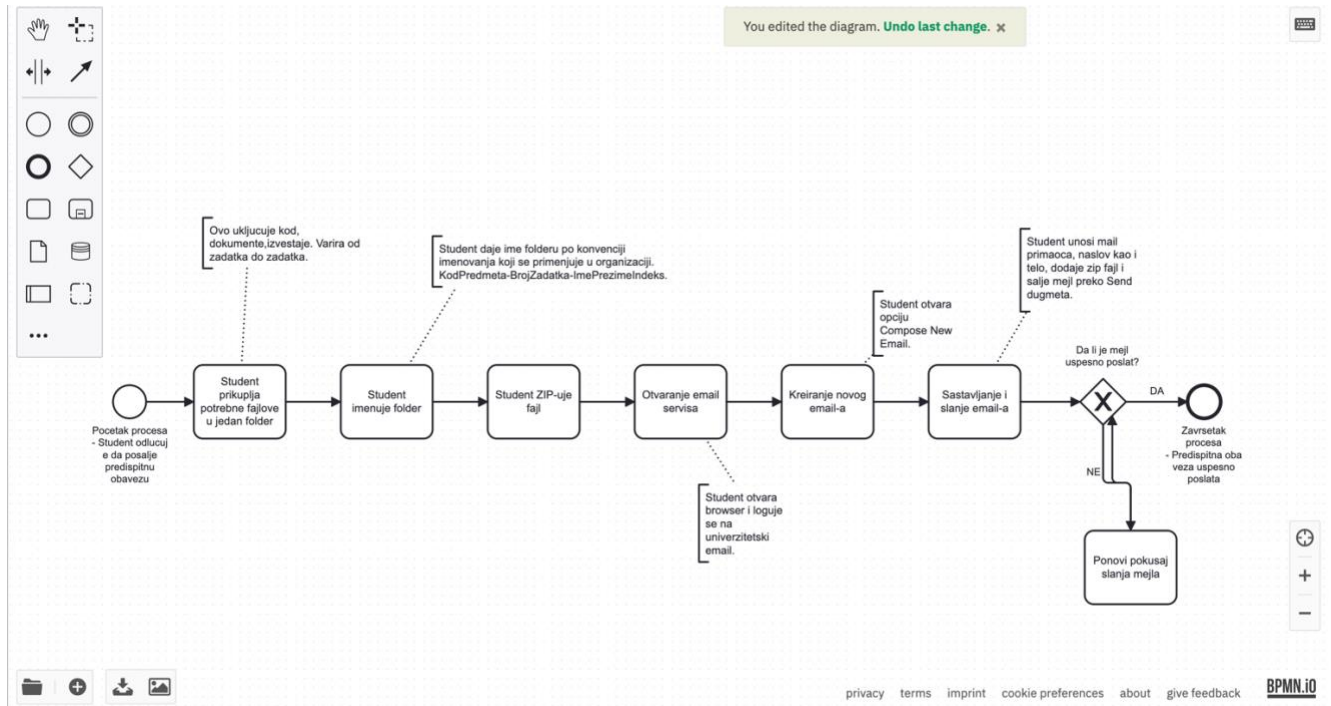
Ovaj proces je veoma ponovljiv jer je slanje predispitnih obaveza putem mejla asistentima obavezan za svaki predmet, sto je ključno za studentovo napredovanje i ostvarivanje biznisa pružanja edukacije.

Slanje predispitnih obaveza je proces koji ima jasan pocetak, gde student manuelno zapocinje sastavljanje doslednih fajlova ili dokumenata koji su potrebni za predispitnu obavezu, a završava se isporucivanjem istih putem mejla asistentu. Takodje ovaj proces je merljiv jer se jasno mogu metrikama videti njegovi rezultati i performanse, poput vremena slanja, stopa gresaka...

Sto se tice tipa poslovnog procesa, ovaj poslovni proces je operativni proces. On je direktno vezan za osnovnu funkciju organizacije, sto je u ovom slucaju obrazovanje, jer studenti kao klijenti koriste usluge nastavnog osoblja a slanje samih predispitnih obaveza je deo tog toka. Samim tim sto se ovim procesom daje neometano obrazovanje, ono održava studente koji placaju za isto, i time indirektno ostvaruje doprinos sto ga čini operativnim procesom.

Sa obzirom da je proces eksterni, ponovljiv, fleksibilan, ima definisan pocetak i kraj, kao i merljivost, mi mozemo zakljuciti da proces slanja predispitnih obaveza, po karakteristikama potpuno spada u realan poslovni proces. Nakon sto smo definisali poslovni proces ove studije slucaja, treba ga i modelovati, a kasnije i opisati njegov problem koji resavamo automatizacijom.

3.2 BPMN Model



Slika 9: BPMN procesa slanja predispitne obaveze.

Preko sajta <https://bpmn.io/> je napravljen tok poslovnog procesa slanja predispitnih obaveza prosečnog studenta, ugledajući se na BPMN model. Kao već spomenuto ovo se koristi radi lakše vizualizacije samog procesa i aktivnosti koje se desavaju pri samom procesu.

Korisceni su graficki elementi koji su spomenuti u teoretskom delu.

Imamo na pocetku start event krug koji nam pokazuje pocetak procesa i da se student odlucuje da posalje predispitnu obavezu.

Odmah nakon toga imamo niz activity objekata koji pokazuju koje aktivnosti student manuelno preuzima kako bi izvršio svoj cilj slanja predispitne obaveze. Povezani su normalnom strelicom koja pokazuje sekvencijalni tok aktivnosti. Prva jeste prikupljanje svih fajlova, kodova, dokumenata i ostalog sto je korisceno u izradi predispitne obaveze. Druga je da se folder nazove po propisanom standardu organizacije za nazivanje a to je KodPredmeta-BrojZadatka-ImePrezimelndeks. Treća je ZIP-ovanje samog foldera. Četvrta jeste otvaranja browsera i logovanja na univerzitetski mejl. Peta je kreiranje novog mejla, dok se u sesto sastavlja ceo mejl, sto ukljucuje adresu primaoca, naslov, telo kao i predispitni folder.

Zatim nailazimo na diamond gateway posle slanja koji proverava da li je uspesno poslato, ako jeste i primi se poruka od servera ide se na event krug koji znaci zavrsetak procesa i uspesnost, a ako nije zbog greske koju server dodeljuje, koja moze biti do pogresne adrese, loseg fajla... ide se na activity koji ponavlja pokusaj slanja mejla pa se opet ide na gateway i proverava da li je uspesno poslato.

Ovaj model nam daje jasnu sliku aktivnosti, korak po korak, koju trebamo automatizovati u studiji slucaja.

3.3 MuleSoft Anypoint Studio i MuleSoft

Za izradu automatizacije ovog projekta i studije slučaja, izabrao sam MuleSoft-ov Anypoint Studio, primarno jer smo ga najviše radili na ovom predmetu, sa ciljem da se bolje upoznam sa IDE-em i njegovim mogućnostima kao i opcijama.

Ova studija slučaja je mogla da se izvrši preko dosta drugih okruženja i alata koji su i pogodniji kao i bolji za ovaj slučaj, ali iz svog zanimanja i eksperimenta zeleo sam da vidim kako bi se to implementiralo u MuleSoft okruženju i to mi je bio primarni cilj.

Sam MuleSoft jeste vodeća platforma za integraciju koji služi organizacijama da povezu svoje podatke, aplikacije i uređaje bez obzira gde se oni nalaze da li to bilo u cloud-u, internim serverima ili hibridnim okruženjima. Kompanije koriste različite softvere poput CRM, ERP, baze podataka, SaaS servise i mnoge druge, gde MuleSoft pruža alate i tehnologije da se svi ti različiti sistemi povezu kako bi mogli da razmenjuju podatke i rade zajedno.

Anypoint Studio je IDE, tačnije graficko razvojno okruženje koje je zasnovano na Eclipse platformi. Specijalno je dizajnirano za developere kako bi mogli da projektuju, razvijaju i testiraju Mule aplikacije, API-je i integracije koje implementiraju.

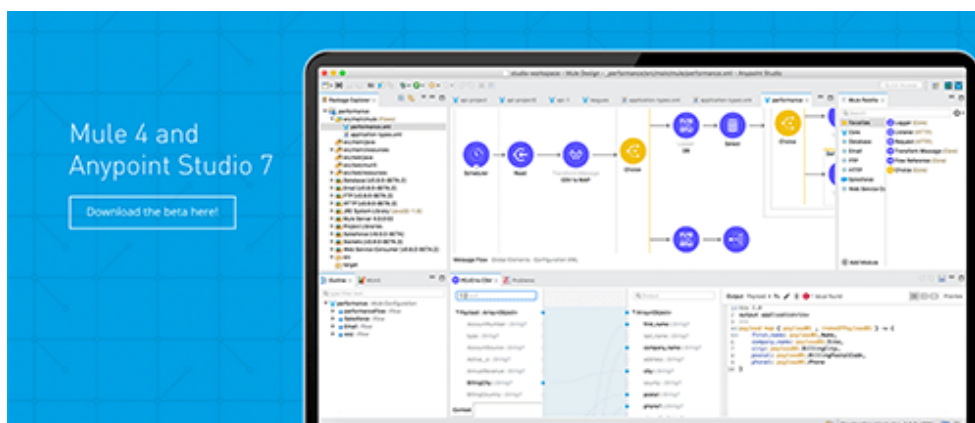
Jedna od glavnih karakteristika Anypoint Studio-ja je ta što ima vizuelni dizajner ili flow designer, što omogućava drag and drop pristup pri kreiranju tokova. Znaci komponente koje sadrži kao što su konektori, transformatori, ruteri i ostali moduli se prevlače na radnu površinu i povezuju na različite načine kako bi se oformila logika obrade i tok podataka. Sama ova apstrakcija dosta olakšava razumevanje kompleksnih integracija.

Iako nudi vizuelni dizajner i interfejs, u pozadini se u svakom projektu kreira konfiguracioni fajl u XML formatu. Sto pruža veću versitilnost i omogućava lakše konfigurisanje u slučaju da vizuelni deo ne pruža neke opcije, što korisnicima daje veću kontrolu.

Takodje Anypoint sadrži i Mule Runtime što omogućava developer-ima da pokreću i testiraju svoje aplikacije na svojoj masini pre deploy-ovanja. Mule je skalabilan, lagan i Java bazirana platforma za integraciju. Isto sadrži i jako mocan i dobar debugger koji pomaze dosta u otklanjanju gresaka u samom flow-u.

DataWave je jezik za transformaciju podataka specifičan za MuleSoft koji omogućava lako mapiranje i transformaciju podataka između različitih formata kao što su JSON, XML, CSV, Java Object, EDI.

Anypoint isto pruža direktan pristup Anypoint Exchange što je MuleSoft-ov repozitorijum gde developeri mogu da pretražuju i koriste predefinisane konektore, template-ove, API-je...



Slika 10: Primer izgleda Anypoint Studio-a.

3.4 DEMO



Video 1: Video snimak demonstracije kako pokrenuta automatizacija radi.

U gore priloženom demo video snimku mozemo videti implementiranu automatizaciju spomenutog poslovnog procesa u nasoj studiji slucaja.

Ono sto je glavno jeste da imamo AUTOMATION folder koji je direktno povezan sa Anypoint aplikacijom i služi kao glavni ulaz za automatizaciju.

Glavna ideja samog projekta jeste da korisnik, nakon sto je aplikacija pokrenuta, jedino sto ima da uradi jeste da napravi folder sa zeljenim predispitnim sadrzajem, nazove ga po konvenciji imenovanja na fakultetu sto je oznakaPredmeta_tipPredispitne_brojPredispitne, za predmete koje ima u ovom semestru i koji su definisani u bazi/json fajlu u samoj aplikaciji, i ubaci taj svoj fajl u AUTOMATION folder.

Kao i sto se vidi na snimku nakon toga se iz AUTOMATION folder izbacuje prvobitno uneti fajl, zatim njegova zipovana verzija, dok se u Anypoint-u ispisuju odgovarajuci logovi, na zeljeni mejl (u ovom slucaju sam slao na svoj mejl ali inace bi primalac bio asistent na tom predmetu) stize generisani mejl sa tacnom porukom koja opisuje koja se predispitna obaveza salje i sa Google Drive linkom tog zipovanog fajla. Posle ove operacije AUTOMATION folder je slobodan da primi novi fajl jer dokle god je aplikacija ukljucena on osluskuje za novi fajl, gde mozemo poslati razlicitu predispitnu obavezu.

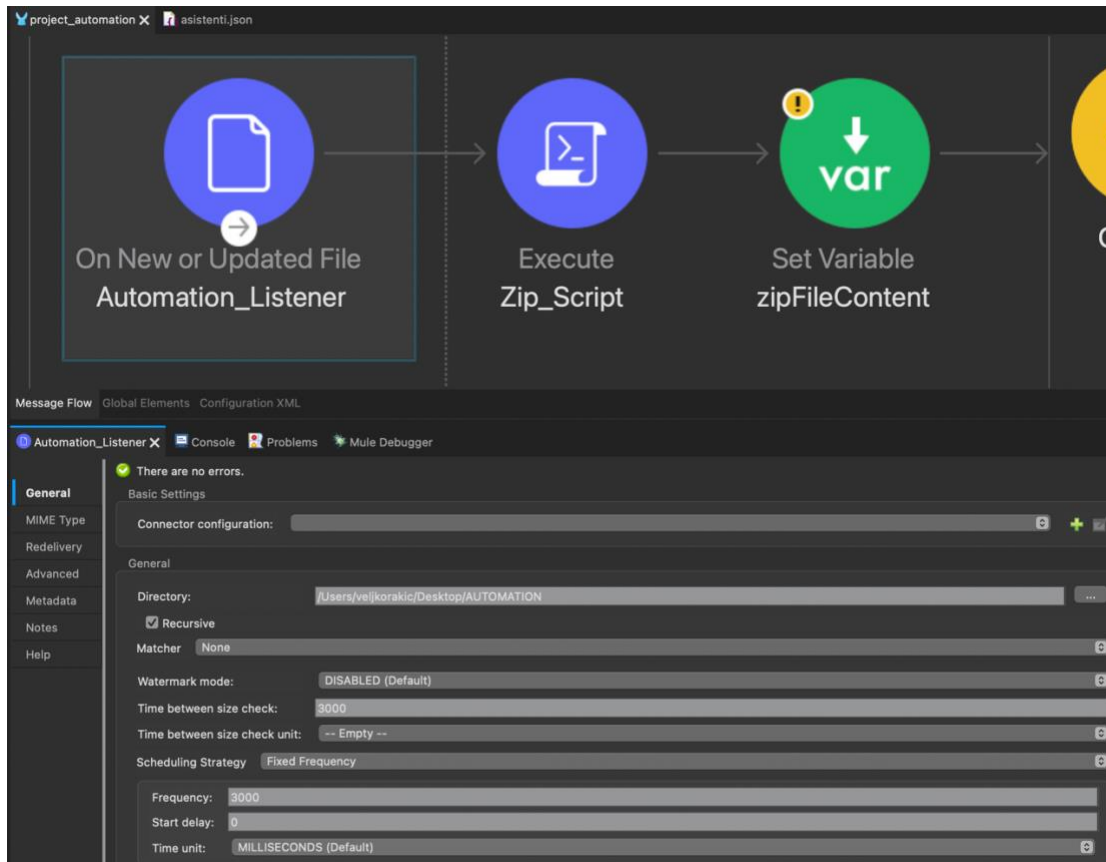
Ova automatizacija kao sto mozemo videti smanjuje broj koraka iz naseg BPMN modela sa 7 kompleksnijih manualnih koraka na cak 3 jako prosta, sto su sakupljanje fajlova, imenovanje fajla i prevlacenje u AUTOMATION folder.

Ova aplikacija bi se na racunarima ili uredjajima koji koristi neka kompanija, mogla implementirati kao lightweight pozadinska aplikacija, sto bi doprinelo smanjenju trosenja vremena svih zaposlenih, kao i povecanju produktivnosti.

Ovo je sama zamisao i implementacija automatizacije poslovnog procesa koji smo imali kao zadatak u studiji slucaja, a u sledecem delju cemo detaljno proci kroz Mule aplikaciju, ceo njen tok i objasniti sve komponente i karakteristike.

3.4 Tehnicke specifikacije aplikacije za automatizaciju

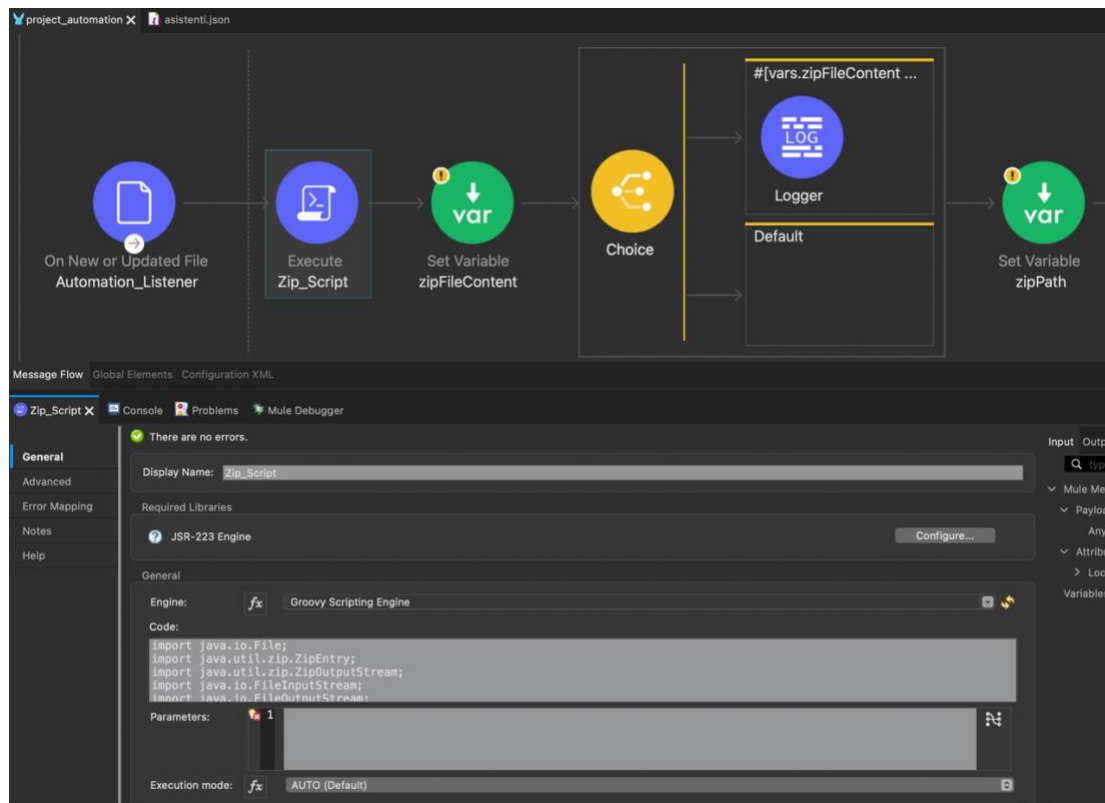
U ovom delu cemo proci kroz ceo tok aplikacije u Anypoint Studio-u, objasnjavajući kako i za sta služi upotrebljeni model ili implementacija.



Slika 11: On New or Updated File.

Prvi modul sa kojim se susrecemo u Mule toku jeste On New or Updated File, pod nazivom Automation_Listener, koji je source konektora u File Connector-u Anypoint-a i ovaj modul služi za iniciranje toka obrade, tacnije Mule flow-a. On kreće iniciranje kada se kreira nova datoteka ili kada se postojeća datoteka azurira u specifičnom direktorijumu, u ovom primeru smo stavili da prati direktorijum AUTOMATION, koji sada služi kao source point i osluskivac.

Ovaj modul u određenim intervalima, što smo stavili na 3000 milisekundi ili 3 sekunde u našem slučaju, proverava navedeni direktorijum za nove ili izmenjene datoteke. Ovo smo namestili preko Scheduling Strategy koju smo stavili na Fixed Frequency, druga opcija jeste i Cron Expression. Znači ovim modelom pocinjemo svoj tok podataka kao i prosledjivanje svih podataka u process delu gde se vrše sve glavne izmene.



Slika 12: Prvi deo process dela toka, Execute modul za zipovanje kao i moduli za Choice i Set Variable.

Nakon sto smo postavili da nam AUTOMATION folder bude osluškivac i source naseg toka, svi fajlovi se prenose u process deo toka. Oni odma nailaze na Execute modul Zip_Script iz Scripting modula, koji služi za njihovo zipovanje kao i prenos tih zip-ovanih podataka i postavljanje njih kao varijable u Set Variable modul zipFileContent. Takodje se path ka tom novo zip-ovanom fajlu postavlja u Set Variable zipPath koje cemo sve objasniti nakon sto objasnimo Zip_Script modul.

Kao sto smo rekli Zip_Script služi za zipovanje fajlova koji su prosledjeni iz Automation_Listener-a, gde on takodje i salje potrebene podatke iz zip-ovanja u ostatak toka. Pre svega treba reci da sam se odlucio za skriptu jer module koje je Anypoint davao i koje sam ubacivao preko Exchange-a nisu zadovoljavali potrebe, tako da mi je ovo bio kako laksi tako i prakticniji nacin.

Za scripting engine i jezik sam se odlucio za Groovy. Groovy je dinamički programski jezik za JVM, koji je razvijen sa ciljem da bude produktivniji i laksi za koriscenje od Java-e, a da ima potpunu kompatibilnost sa Java ekosistemom. On kombinuje dobre karakteristike OOP-a sa konceptima iz dinamičkih jezika kao sto je Python i Ruby. Takodje se ovaj jezik dosta primenjuje u scripting-u, tako da je bio lak izbor.

```

7 import java.nio.file.StandardCopyOption;
8 import java.util.HashMap;
9 import java.util.Map;
10
11 def result = new HashMap<String, Object>();
12
13 try {
14     def folderName = attributes.fileName;
15     def folder = new File("/Users/veljkorakic/Desktop/AUTOMATION/" + folderName);
16     def automationFolder = new File("/Users/veljkorakic/Desktop/AUTOMATION");
17
18     if (folder.isDirectory()) {
19         def zipFilePath = folder.getParentFile().getAbsolutePath() + "/" + folderName + ".zip";
20         def zipFile = new File(zipFilePath);
21
22         if (!zipFile.exists()) {
23             def filteredFiles = folder.listFiles().findAll { !it.name.startsWith(".DS_Store") };
24
25             if (filteredFiles.size() > 0) {
26                 try (FileOutputStream fos = new FileOutputStream(zipFile);
27                     ZipOutputStream zos = new ZipOutputStream(fos)) {
28
29                     for (File file : filteredFiles) {
30                         if (file.isFile()) {
31                             try (FileInputStream fis = new FileInputStream(file)) {
32                                 ZipEntry zipEntry = new ZipEntry(file.getName());
33                                 zos.putNextEntry(zipEntry);
34
35                                 byte[] bytes = new byte[1024];
36                                 int length;
37                                 while ((length = fis.read(bytes)) >= 0) {
38                                     zos.write(bytes, 0, length);
39                                 }
40                             }
41                         }
42                     }
43                 }
44             }
45         }
46
47         def desktopPath = System.getProperty("user.home") + "/Desktop";
48         def desktopFolder = new File(desktopPath);
49         def destinationFolder = new File(desktopFolder, folder.getName());
50
51         if (!folder.renameTo(destinationFolder)) {
52             result.put("zipFilePathFromScript", null);
53             result.put("zipFileContent", null);
54         } else {
55             result.put("zipFilePathFromScript", zipFilePath);
56
57             result.put("zipFileContent", Files.readAllBytes(zipFile.toPath()));
58         }
59
60         def signalFile = new File(automationFolder, "zip_finished.txt");
61         signalFile.createNewFile();
62     }
63 } catch (Exception overallException) {
64     println "Dosló je do greske u skripti: " + overallException;
65     overallException.printStackTrace();
66     result.put("zipFilePathFromScript", null);
67     result.put("zipFileContent", null);
68 }
69
70 return result;
71

```

Slika 13: Zip_Script Groovy skripta za zipovanje fajla.

Na slici 13 mozemo videti ceo kod skripte za zipovanje fajla. Cilj skripte je da pronadje folder unutar direktorijuma AUTOMATION, zip-uje ceo folder, premesti originalni folder nazad na Desktop korisnika i vrati putanju do kreirane zip datoteke kao i njen sadrzaj, na kraju takodje kreira i signal datotetku da oznaci zavrsetak procesa.

Na samom pocetku koda imamo potrebne import-e iz java. koja uvozi razne klase, interfejsse i enumere.

Na pocetku `def result = new HashMap<String, Object>();` kreira praznu i novu HashMap objekat result. Ova mapa ce se koristiti za cuvanje izlaznih vrednsti skripte koje ce biti vracene na kraju izvršavanja sto vidimo na kraju sa `return result;`. Cuvaju se kao kljucevi koji su String tipa (npr. "zipFilePathFromScript") a vrednosti mogu biti bilo kog Object tipa sto je u nasem slucaju ili putanja do fajla kao String ili sadrzaj fajla kao byte[].

Nakon toga ostatak koda je obavljen u try-catch za obradu izuzetaka, gde ako se desi greska iz try se prelazi na catch koji hvata tip izuzetka i izvršava logiku sto je ovde print-ovanje izuzetka i postavljanje null vrednosti u sa funkcijom .put.

Kada udjemo u try prvo na sta nailazim jeste def folderName = attributes.fileName; gde se promenljiva folderName stavlja da bude ime dobijenog fajla preko attributes.fileName.

Sledece def folder = new File("/Users/veljkorakic/Desktop/AUTOMATION/" + folderName) kreira File objekat folder koja nastaje spajanjem fiksne putanje kao i folderName i predstavlja ciljani folder.

Dok def automationFolder = new File("/Users/veljkorakic/Desktop/AUTOMATION"); kreira File objekat koji predstavlja sam AUTOMATION direktorijum.

Zatim nalazimo na if (folder.isDirectory()) { ... } koji proverava da li folder objekat zaista predstavlja folder u fajl sistemu, gde ako predstavlja izvršava se kod unutar tog if-a a ako nije preskace se.

Unutar ovog if-a def zipFilePath = folder.getParentFile().getAbsolutePath() + "/" + folderName + ".zip"; pravi punu putanju za zip fajl koji ce biti kreiran, folder.getParentFile() predstavlja roditeljski direktorijum od foldera sto je /Users/veljkorakic/Desktop/AUTOMATION/, .getAbsolutePath() dobija putanju roditeljskog kao String, dok + "/" + folderName + ".zip" dodaje na roditeljsku putanu / i .zip.

Onda def zipFile = new File(zipFilePath); kreira File objekat za zip datotetku koristeći vec napravljenu putanju kroz zipFilePath.

Nakon ovoga ulazimo u jos jedan if a to je if (!zipFile.exists()) { ... } koji proverava da li zip fajl sa tim imenom vec postoji na toj lokaciji i ako ne postoji izvršava se blok koda za kreiranje zip fajla, gde ovo sprecava da skripta prepise postojeći zip fajl ako se pokrene vise puta za isti folder pre nego sto se on premesti.

Takodje sam uveo i def filteredFiles = folder.listFiles().findAll { !it.name.startsWith(".DS_Store") }; jer je u pocetku je bio problem zato sto macOS u praznim direktorijumima stvara skrivene fajlove DS_Store koje je listener prepoznavao kao fajlove koje treba preraditi i njih zipovao tako da sam uveo ovaj filter da prosledjuje fajlove u filteredFiles cije ime ne pocinje sa DS_Store.

Sledeci if proverava da li promenljiva filteredFiles ima bar jedan fajl nakon filtriranja i ako ima nastavlja sa kreiranjem zip fajla.

Nakon toga imamo try-with-resources sto je robustan nacin za rad sa resursima, I/O stream-ovima u ovom slucaju i sprecava curenje resursa jer se na kraju try-a odma zastvaraju.

FileOutputStream fos = new FileOutputStream(zipFile) otvara izlazni stream za pisanje u zipFile, dok ZipOutputStream zos = new ZipOutputStream(fos) kreira ZipOutputStream koji ce pisati kompresovane podatke u fos.

U for (File file : filteredFiles) { ... } se iterira kroz svaki File objekt u listi filteredFiles, a if (file.isFile()) { ... } proverava da li je trenutni element file zaista fajl a ne poddirektorijum jer ih ova skripta ne obradjuje samo fajlove na prvom nivou.

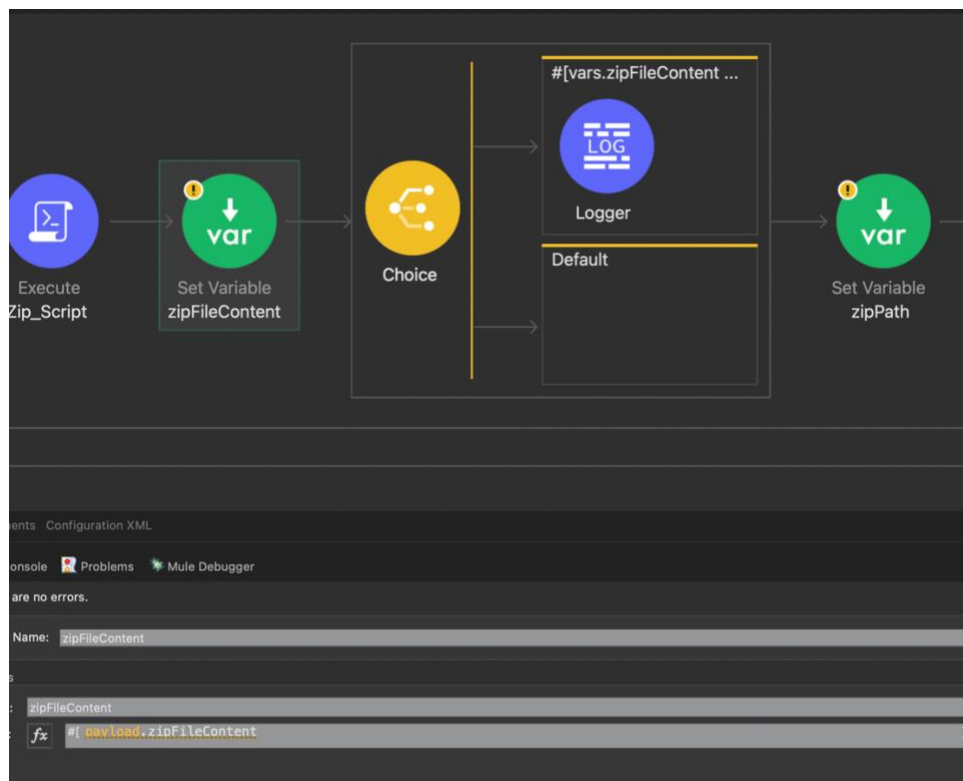
Opet u try-with-resources, try (FileInputStream fis = new FileInputStream(file)) { ... } otvara ulazni stream za citanje trenutnog file. ZipEntry zipEntry = new ZipEntry(file.getName()); kreira novu promenljivu za zip arhivu i kao ime unosa koristi originalno ime fajla. Dok zos.putNextEntry(zipEntry); dodaje zipEntry u zos pripremajući ga za pisanje podataka zipEntry-a.

Sada prelazimo na pisanje citanje i pisanje bajtova ge byte[] bytes = new byte[1024]; kreira bajt niz koji služi kao bafer za citanje podataka iz fajla, a int length; cuva broj procitanih bajtova u svakom koraku.

Ova while ((length = fis.read(bytes)) >= 0) {zos.write(bytes, 0, length);} petlja cita podatke iz fis u bytes i pise ih u zos. Gde nakon zavrsetka while petlje ceo fajl je procitan i upisan unutarasnji try-with-resources zatvara fis dok se zos.closeEntry() se implicitno poziva kada se zos.putNextEntry() pozove sledeci fajl ili kada je zos zatvori sa spoljnim try-with-resources.

Sledeci desktopPath dobija dinamicki putaju do korisnickog home direktorijuma i na nju dodaje /Desktop, dektopFolder kreira file objekat koji predstavlja Dekstop direktorijum, destinationFolder kreira File objekat koji predstavlja ciljnu lokaciju na Desktopu. Nakon toga if pokusava da sa renameTo premesti orginalni folder na Desktop i ako ne uspe on stavlja sve ostalo za HashMap-u kao null. A ako uspe stavlja u result mapu i zipFilePath kao putanju do kreiranog zip fajla kao i sto cita ceo sadrzaj kreiranog zip fajla kao niz bajtova i stavlja ih pod kljucem zipFileContent.

Na kraju imamo pravljenje signalnog fajla kojeg ce druga skripta koristiti, obradu gresaka u catch bloku kao i vracanje rezultata u return result.

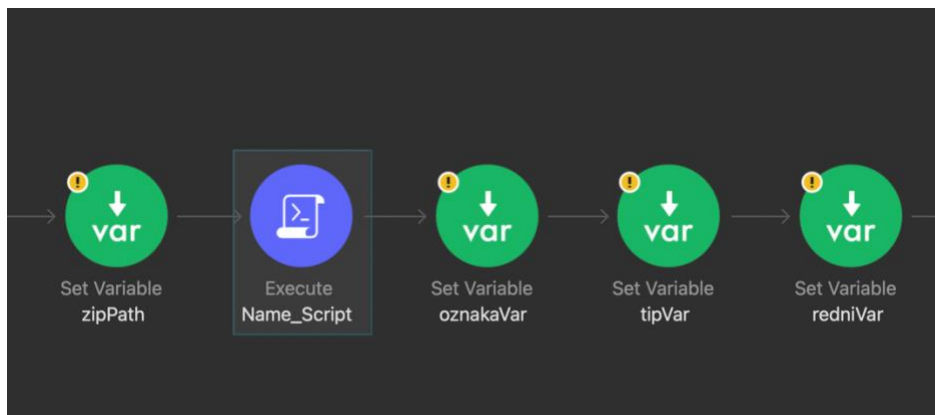


Slika 14: Set Variable i Choice.

Nakon uspesnog završavanja Zip_Script-e, sve sto je bilo vraceno i sacuvano sa return results je smesteno u payload ovog Execute modula. Sa sledecim modulom Set Variable postavljamo varijablu koja cuva ono sto unesemo u polje Value. U ovom slucaju u Set Variable zipFileContent mi stavljam Value da bude DataWave izraz payload.zipFileContent sto referencira zipFileContent iz HashMap gde se nalazi sadrzaj zip-ovanog fajla u bajtovima.

Nakon toga imamo Choice modul koji prima ekspresiju koja ako je tacna izvršava ono sto je u When polju, u ovom slucaju modul Logger-a koji ispisuje poruku da je automatizacija pokrenuta. Ovaj Choice modul sam stavljam u projektu da bi preventirao prosledjivanje null vrednosti jer kao sto smo rekli listener je podesen na frekvenciju od 3000 milisekundi i ako se nista ne ubaci on salje null vrednosti, sto bi remetilo i izbacivalo greske u celom toku. Mana ovoga u Anypoint-u sto nema striktno even-based listener ali smo presli ovo uz pomoc Choice modula. Generalno u ovom slucaju je Expression bio vars.zipFileContent != null.

Nakon toga imamo jos jednu Set Variable-u a to je zipPath koja u payload-u cuva payload.zipFilePathFromScript.



Slika 15: Name_Script skripta i njene varijable..

```
import java.io.File

def result = [:]

try {
    def automationFolder = new File("/Users/veljkoracic/Desktop/AUTOMATION");

    def maxWaitTime = 10000;
    def startTime = System.currentTimeMillis();
    def signalFile = new File(automationFolder, "zip_finished.txt");

    while (System.currentTimeMillis() - startTime < maxWaitTime) {
        if (signalFile.exists()) {
            signalFile.delete();
            break;
        }
        Thread.sleep(1000);
    }

    def zipFiles = automationFolder.listFiles().findAll { it.name.endsWith(".zip") };

    if (zipFiles.size() > 0) {
        def zipFile = zipFiles[0];
        def zipFileName = zipFile.name;

        def fileNameWithoutExtension = zipFileName.replace(".zip", "").trim();
        def parts = fileNameWithoutExtension.split("_");

        if (parts.size() == 3) {
            def oznakaPredmetaStr = parts[0];
            def tipPredispitneStr = parts[1];
            def redniBrojInt = parts[2]?.toInteger();

            result.oznakaPredmetaIzPayload = oznakaPredmetaStr
            result.tipPredispitneIzPayload = tipPredispitneStr
            result.redniBrojIzPayload = redniBrojInt

        } else {
        }
    } else {
        println "Nije pronadjen zip fajl u AUTOMATION folderu.";
    }
} catch (Exception e) {
    println "Došlo je do greške u Extraction_Script: " + e.getMessage();
    e.printStackTrace();
}

return result
```

Slika 16: Kod Name_Script-e.

Cilj ove druge skripte jeste da saceka signalnu datoteku koju kreira Zip_Script-a, pronadje zip datoteku u AUTOMATION folderu, analizira ime te datoteke kako bi izvukla oznaku predmeta, tip predispitne, redni broj iz njenog imena i na kraju vrati te izvucone informacije u obliku mape kao i kod prve.

Opet se na pocetku koristi HashMap za cuvanje podataka sa imenom result.

Isto ceo kod obuhvata try-catch radi gresaka.

Isto se definise putanja do AUTOMATION folder sa promenljivom automationFolder.

Ovde je i vreme postavljeno sa maxWaitTime i startTime kako bi se dalo vremena da prva skripta završi svoj deo posla. A signalFile definise File objekat za signalnu datoteku koju treba cekati.

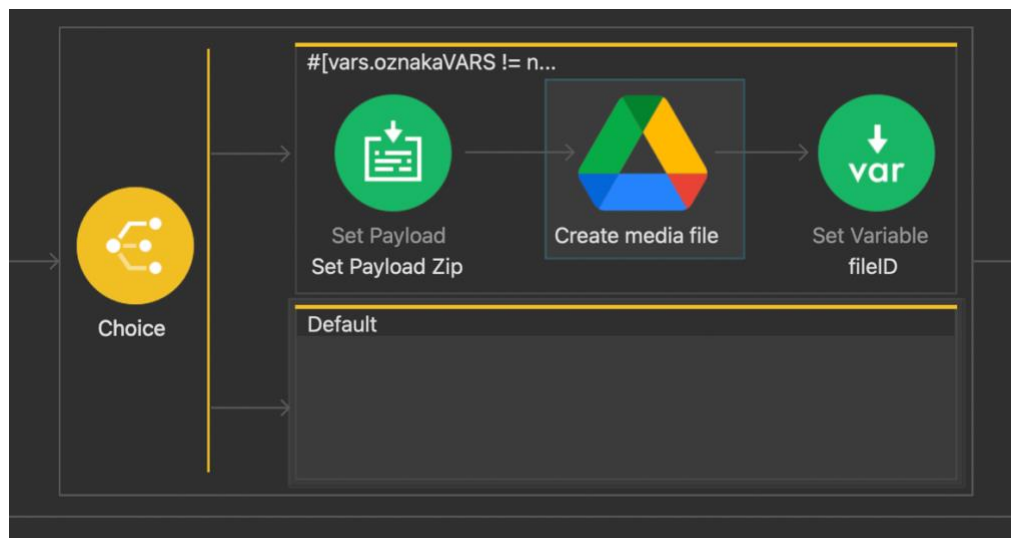
While petlja se izvršava dok ne istekne maxWaitTime gde se proverava da li signalna datoteka postoji i ako postoji brise je, dok break prekida petlju ako je signal pronadjten.

Promenljiva zipFiles pregleda sve stavke u automationFolder i izdvaja samo one koji se završavaju sa .zip.

Sledeca if naredba proverava da li ima bar jedan .zip fajl i ako ima uzima prvu iz liste, ali ako nije vec spomenuto, u ovaj folder za automatizaciju treba da se ubacuju fajlovi tek nakon sto je jedan potpuno obradjen i to jedan po jedan. Ime fajla se dobija u zipFileName, gde se posle sa trim skida .zip ekstenzija, a ostatak imena se deli na delove koristeći _ kao separator.

Nakon ove sledeca if naredba izvalci podatke iz delova i smesta ih u results HashMap-u koja direktno to smesta kao payload.

Ovi podaci su dalje smesteni u tri Set Variable oznakaVar, tipVar, redniVar. Njima se pristupa sa payload. i u njih se smestaju oznaka predmeta, tip predispitne kao i redni broj predispitne.



Slika 17: Upload-ovanje zip-ovanog fajla na Google Drive.

Sledeci korak nakon sto smo osigurali zipovanje potrebnog fajla kao i ekstrakciju informacija iz njegovog imena jeste njegove upload-ovanje na Google Drive. Prvo smo stavili Choice modul da ne moze prihvatiti niti jednu varijablu sa null vrednoscu vars.oznakaVARS != null and vars.tipVARS != null and vars.redniVARS != null and vars.zipFileContent != null. Zatim zato sto koristimo simple send na modul Create media file za uploadovanje, mi u njegov body mozemo staviti samo payload, zbog toga smo pre ovo modula stavili modul Set Payload sa Value vars.zipFileContent, sto predstavlja podatke naseg zip-ovanog fajla.

Sada prelazimo na Create media file koji je modul Google Drive Connector-a koji sam uvezao u Anypoint preko Exchange-a. On pruža različite module i konektore koji rade sa Google Drive-om, a ovaj konkretno služi za upload-ovanje bilo kakve media.

There are no errors.

Display Name: Create media file

Basic Settings

Connector configuration: Google_Drive_Connector_Config

General

Supports All Drives: False (Default)

Ignore Default Visibility: False (Default)

Enforce Single Parent: False (Default)

Pretty Print: False (Default)

Keep Revision Forever: False (Default)

Use Content As Indexable Text: False (Default)

Quota User: fx

key: fx

User Ip: fx

Oauth token: fx

Include Permissions For View: fx

alt: fx -- Empty --

fields: fx

Ocr Language: fx

Supports Team Drives: False (Default)

Content Type: fx application/zip

Content Length: fx

Body: fx 1 payload

Slika 18: Konfiguracije Create media modula.

Ovde se mogu videti proste konfiguracije koje su uglavnom ostavljene na default, Content Type je postavljen na application/zip da bi MuleSoft ili Anypoint znao koji je tip podataka koji šaljem kako bi pravilno obradio i prosledio ga Google Drive API-ju. Za body smo ostavili payload kako bi preuzeo iz prethodnog Set Payload-a.

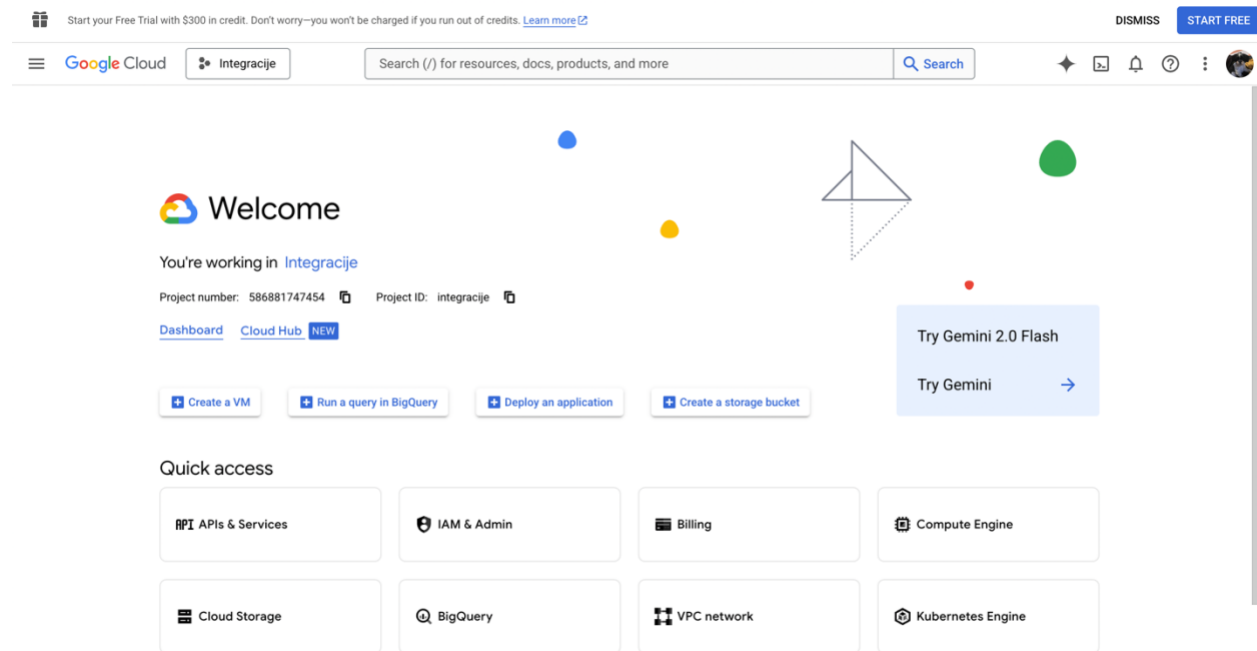
Sledeće što ćemo obraditi je Connector configuration koji postavljamo za konekciju između naše MuleSoft aplikacije i Google Drive-a. Ona služi za uspostavljanje veze kao i što sadrži informacije i podešavanja neophodna da bi MuleSoft mogao se autentifikovati i razmenjuje sa Google Drive API-jem.

Ali pre nego što objasnimo podešavanja ovih konfiguracija, treba da objasnimo standardni protokol za autorizaciju koju Google Drive koristi kako bi nam dao autorizaciju da koristimo ove module sa našim Google Drive nalogom. Google Drive koristi OAuth 2.0 što je industrijski standardni protokol autorizaciju i on omogućava third-party aplikacijama, što je u naša aplikacija, da dobiju ograničen pristup korisničkim nalogima na veb servisima, bez upotrebe da im se otkriju korisničke lozinke. On ukratko radi na principu delegiranja autorizacije.

Koncepti u OAuth 2.0 koje ćemo odmah posle opisati kako smo podešili su vlasnik resursa koji poseduje podatke i može da odobri pristup tim podacima. Klijent tačnije aplikacija koja želi da pristupi resursima vlasnika resursa. Server resursa koji host-uje zaštićene resurse što je Google Drive server. Server za autorizaciju koji izdaje pristupne tokene klijentu nakon što dobije saglasnost od vlasnika. Saglasnost vlasnika se vrši sa ID-jevima koji se dobijaju kao kredencijali. Access token je kratkoročna kredencijala koju klijent koristi za pristup zaštićenim resursima na

serveru. Imamo i opseg koji definise nivo pristupa. Redirect URI sto je URL na koji se korisnik preusmerava nakon sto odobri ili odbije zahtev za autorizaciju.

Ovo su osnovne karakteristike i koncepti OAuth 2.0 autorizacije koji se svuda drugacije implementiraju, gde kod nas je bitno samo u konfiguraciji namestiti, sto cemo pokazati, ono sto je potrebno i automatizovano se sve samo obavlja.



Slika 19: Kreiran projekat u Google Cloud Console.

Prvo sto radimo jeste da odemo na Google Cloud Console gde cemo se prijaviti na nas test nalog, na koji takodje i saljemo predispitne. Nakon prijavljivanja pravimo novi projekat u kome cemo omoguciti OAuth 2.0. Odmah nakon toga cemo ici na Credentials meni gde cemo napraviti novi Credentials.

Ali pre nego sto se napravi Credentials moracemo da podesimo OAuth Consent Screen gde cemo za User Type staviti External da je omogucena svima za testiranje jer u ovoj studiji nismo deo organizacije. Zatim popunjavamo App Name i User Support email i Developer contact information. U poljima za Scoops ostavljamo kakva jesu jer nisu potrebna za ovaj projekat. Za test user stavljamo mejl odakle saljemo nasu predispitnu.

Start your Free Trial with \$300 in credit. Don't worry—you won't be charged if you run out of credits. [Learn more](#)

Google Cloud Integracije Search (f) for resources, docs, products, and more Search

Google Auth Platform / Clients / Client: 586881747454-ac1h8nnrb8t5i10p3lpg5mdmss8r9od.apps.googleusercontent.com

Overview
Branding
Audience
Clients
Data Access
Verification Center

Client ID for Web application

Name *
Web client 1

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen as authorized domains](#).

Authorized JavaScript origins

For use with requests from a browser

URIs 1 *
http://localhost:8081

+ Add URI

Authorized redirect URIs

For use with requests from a web server

URIs 1 *
http://localhost:8081/callback

+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Save Cancel

Additional information

Client ID	586881747454-ac1h8nnrb8t5i10p3lpg5mdmss8r9od.apps.googleusercontent.com
Creation date	April 12, 2025 at 9:40:18 PM GMT+2

Client secrets

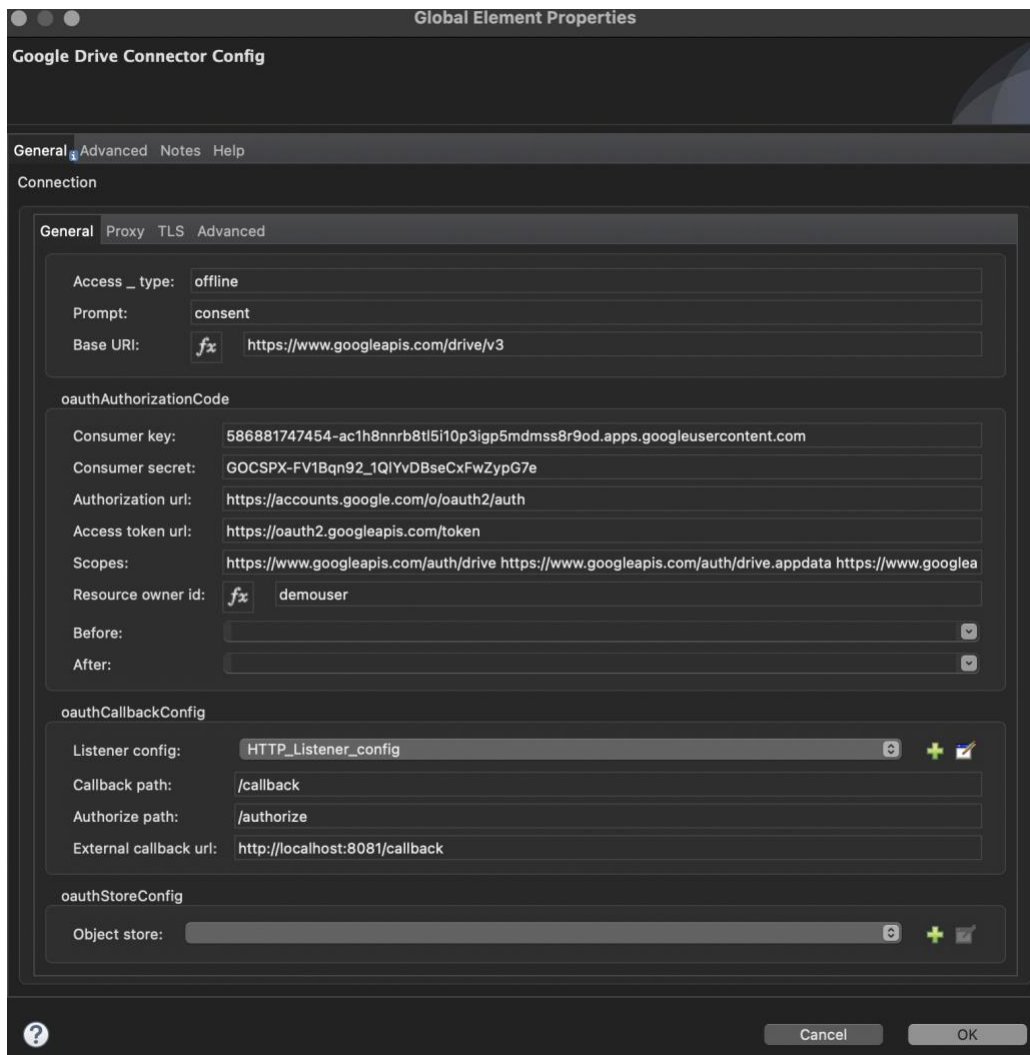
If you are in the process of changing client secrets, you can manually rotate them without downtime. [Learn more](#)

Client secret	GOCSPX-FV18qn92_1QYv0BseCxwZypG7e
Creation date	April 12, 2025 at 9:40:18 PM GMT+2
Status	Enabled

+ Add secret

Slika 20: Client ID za web aplikaciju.

Nakon što smo to završili vraćamo se na Credentials i idemo na Create OAuth client ID i stavljamo za web application i ime. Za Authorized Redirect URI stavljamo `http://localhost:8081/callback` i idemo na create. Ovo je kreiralo potrebne stvari koje ćemo ubaciti u našu Google Drive Connector Configuration. Posebne dve stvari koje su nam potrebne jesu Client ID i Client secrets.

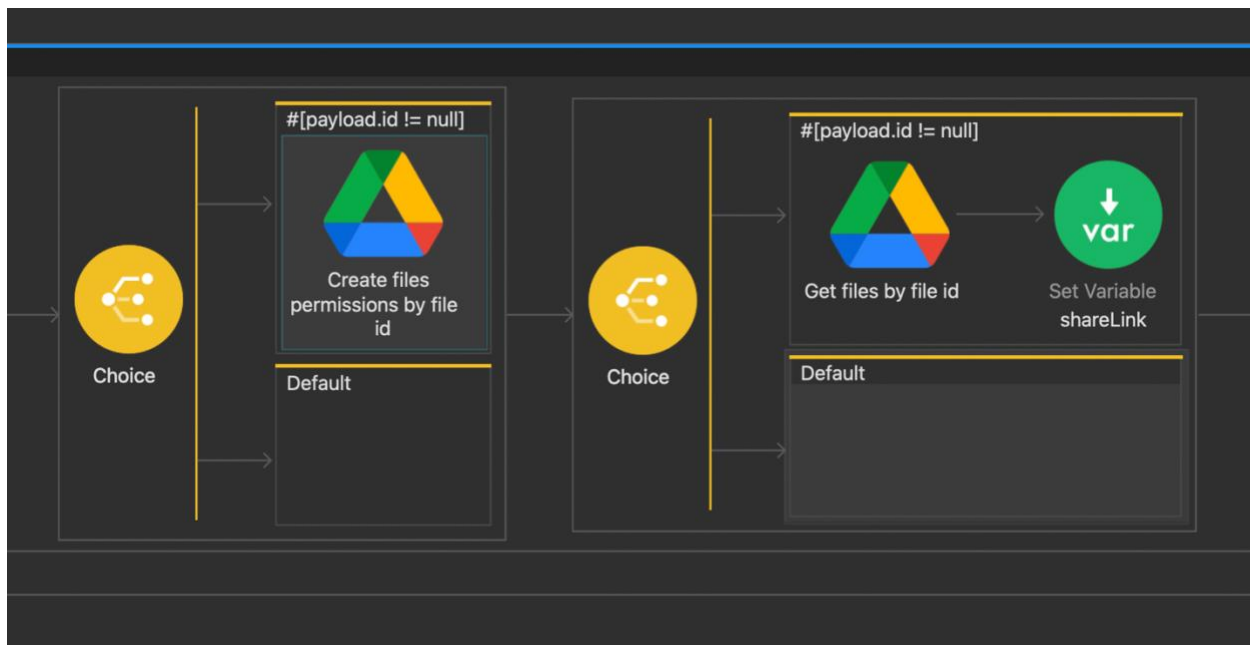


Slika 21: Google Drive Connector Configuration.

U nasim konfiguracijama u Consumer key ubacujemo Client ID sa Google Drive Console, dok u Consumer Secrets ubacujemo Client secrets. Authorization url ostaje isti kakav je bio dok Access token utl menjamo onim kojim dobijamo kada ga skinemo iz json fajla sa Google Drive Console. Scopes ostaju isti i odredjuju privilegije koja nasa aplikacija ima nad Google Drive-om, one se moraju poklapati sa scoop-ovima na Google Drive Console. Za resource owner id koji ce nam posle trebati za autorizaciju cemo staviti samo demouser. Listener config cemo ostaviti kakav jeste globalni element sami port-ovi trebaju da se poklapaju i kao protocol da bude HTTP a host default. Callback path stavljamo na /callback dok Authorize path na /authorize i ovo mozemo pozivati tako sto na <http://localhost:8081> dodamo zeljeni. I na kraju na External callback url postavljamo kao sto smo Authorized Redirect URI.

Da bi se pristup dozvolio trebamo da pokrenemo nasu aplikaciju, odemo na browser i ukucamo <http://localhost:8081/authorize> gde se prijavljujemo na nalog na koji zelimo da damo autorizaciju i cekiramo sve privilegije koje zelimo da damo. Nakon toga trebali bi dobiti poruku u browser-u Successfully retrived access token. Zatim idemo na sledeci url <http://localhost:8081/authorize?resourceOwnerId=demouser> gde smo dodali novi deo ?resourceOwnerId=demouser koji tacno oznacava pre postavljenog resource owner-a, i dajemo istu autorizaciju. Ovime smo uspesno odradili OAuth dance i dali privilegije i pristup nasem modulu da izvrši svoju radnju.

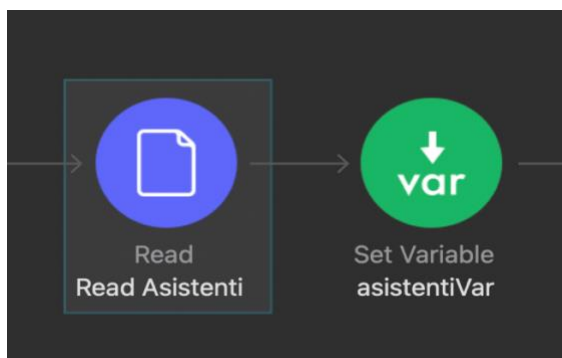
Na kraju onog Choica smo takodje stavili Set Variable za file id koji se generise pri slanju sa Create media file i cuvamo ga pomocu payload.id.



Slika 22: Permission i Get Google Drive moduli.

Nakon što je uspešno upload-ovan zipovan fajl na Google Drive sledeća dva modula su Choice modulima kako `payload.id` iz `fileID` varijable nikad nije null. `Create files permissions by file id` se koristi kako bi se postavili privilegije nad upload-ovanim fajlom, tačnije ko može da mu pristupa i na koji način. Ovaj modul samo zahteva `id` od `fileID` varijable kako bi znao sa kojim fajlom radi i DataWave ekspresiju, što je u ovom slučaju `%dw 2.0 output application/json ---{role: "reader", 'type': "anyone" }` i specificira privilegije.

`Get files by file id` je sličan samo što ima `fields` polje u kojem se specificira šta će preuzeti ovaj modul od tog upload-ovanog foldera, u ovom slučaju `id`, `name`, `webViewLink`, `webContentLink`, gde `webViewLink` je onaj koji nam je potreban jer vraća link ka tom fajlu i ovo čuvamo u `Set Variable shareLink` preko `payload.webViewLink`.



Slika 23: Read modul i Set Variable za asistenti.json.

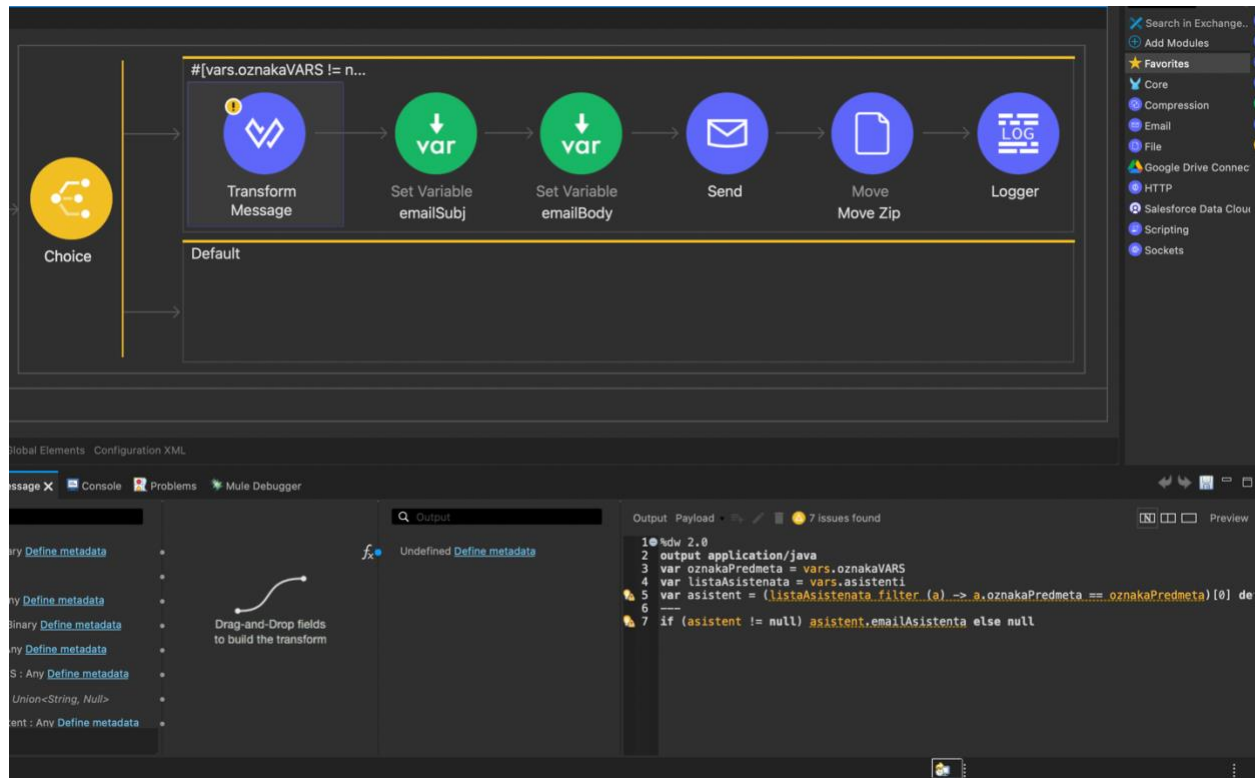
```

1 [
2   {
3     "oznakaPredmeta": "IT380",
4     "emailAsistenta": "veljko1503@gmail.com",
5     "imePredmeta": "Inženjerstvo i integracija sistema"
6   },
7   {
8     "oznakaPredmeta": "CS320",
9     "emailAsistenta": "veljko1503@gmail.com",
10    "imePredmeta": "Operativni sistemi"
11  }
12 ]

```

Slika 24: Asistenti.json.

Ovde imamo Read modul koji služi za čitanje postavljenih fajlova ili vrednosti, on je postavljen na putanji da gadjja asistenti.json koji je json fajl i sadrzi oznake predmeta koji su vezani za mail asistenta i ime predmeta, ovo je samo primer ali bi trebalo imati bazu koja je povezana sa svim predmetima i profesorima. Read fajl cita ovaj json fajl i smesta payload u asistentiVar varijablu.



Slika 25: Transform, Send, Move modul.

Zadnji delovi toka su u Choice modulu cija je ekspresija `vars.oznakaVARS != null and vars.tipVARS != null and vars.redniVARS != null and vars.shareLink != null`. Prvi modul kroz koji se nastavlja tok je Transform modul koji služi DataWave-om i služi za manipulaciju i upravljanje podacima i u našem slučaju ima izraz koji može da se vidi na slici 25 i ona pokušava da pronadje asistenta u listi listaAsistenata koji ima oznakaPredmeta jednaku vrednosti u oznakaPredmeta varijabli, ovo se izvlaci iz prethodnog json-a. Ako pronadje takvog asistenta onda se vraća vrednost iz njegovog polja za emailAsistenta, a ako ne onda vraća null. Izlaz je String što predstavlja mail asistenta.

Set Variable emailSubj i emailBody koriste DataWave i prethodne varijable kako bi oformili tekst u subject mejla i generisali tekst za body mejla ubacivajuci vrednosti koje odgovaraju fajlu.

There are no errors.

Display Name: Send

Basic Settings

Connector configuration: Email_SMTP

Settings

From address: `fx` veljko.rakic.5813@metropolitan.ac.rs

To addresses: Edit inline

Value: `#[payload]`

Cc addresses: None

Bcc addresses: None

Reply to addresses: None

Subject: `fx` `#[ vars.emailSubject ]`

Body

Content: `fx` 1 `vars.emailbody`

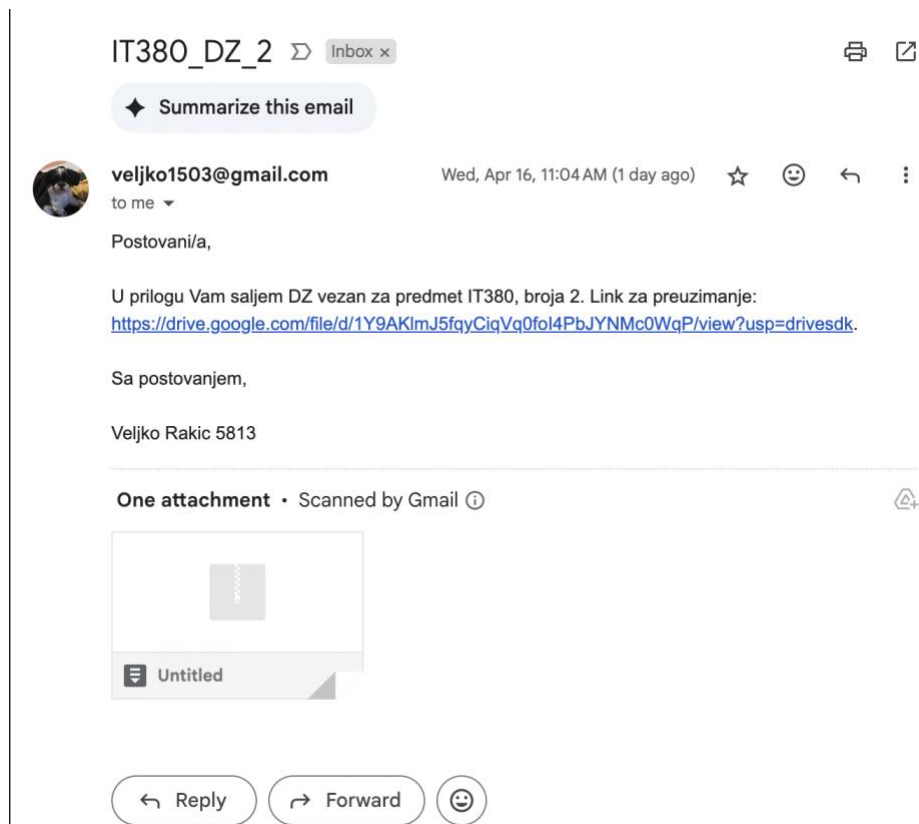
ContentType: `fx` text/plain

Encoding: `fx` UTF-8

Content Transfer Encoding: `fx` Base64

Slika 26: Konfiguracije Send modela.

Na slici se mogu videti konfiguracije Send modela, ovde From address nije tacan jer ga override-uje onaj iz Connector configuration. U njemu je host podesen na smtp.gmail.com, port na 587, user na veljko1503@gmail.com a password je generisan kod od strane Google App Password jer se ne stavlja prava lozinka vec se ova koristi za third-party aplikacije.



Slika 27: Uspesno generisan mejl.

Na slici 27 mozemo videti uspesno generisan mejl pomocu nase aplikacije.

Nakon ovoga u toku je ostao samo Move modul koji premesta zip-ovan fajl iz AUTOMATION foldera na Desktop i Logger koji nam ispisuje da je automatizacija gotova.

4. Zaključak

Ovime smo uspesno završili studije slučaja i pokazali kako se vrsti automatizacija poslovnog procesa od početka pa do kraja. U radu je objašnjena teorija kako sa poslovne strane tako i sa tehnološke a kroz praktičan primer se demonstrirala i primenjenost.

Ova studija slučaja je samo običan primer kako bi trebalo da izgleda ali njoj fali mnogo prover, dodataka i finisa. Kao što sam rekao da se ovo pretovri u stvarno upotrebljivu aplikaciju bilo bi potrebno mnogo više vremena, bolje tehnologije i tehnike višeg stepena da bi mogla da se implementira u pravom okruženju, ali kao studije slučaja i primer za projekat ostvaruje svoj cilj.

5. Literatura

<https://essaypro.com/blog/case-study>

<https://linkversity.hr/blog/kako-napisati-seminarski-rad/>

https://en.wikipedia.org/wiki/Business_process

<https://blog.processology.net/what-is-a-business-process>

<https://www.ef.uns.ac.rs/predmeti/mas/modelovanje-poslovnih-procesa/01ModelovanjePP-2019.pdf>

<https://koris.hr/preuzmi/koris-uvod-u-modeliranje-poslovnih-procesa.pdf>

https://en.wikipedia.org/wiki/Business_Process_Model_and_Notation

<https://alltechbuzz.net/bs/osnove-automatizacije-poslovnih-procesa/>

<https://www.datalab.rs/blog/automatizacija-poslovnih-procesa-kako-da-iskoristite-svoje-vreme-pametnije/>

<https://docs.mulesoft.com/compression-module/latest/compression-documentation>

<https://docs.mulesoft.com/google-drive-connector/latest/>

<https://docs.mulesoft.com/general/>

<https://medium.com/another-integration-blog/mulesoft-google-drive-connector-b19c9bc72e1b>

<https://dashboard.ngrok.com/get-started/setup/macOS>

<https://docs.mulesoft.com/google-drive-connector/latest/google-drive-connector-reference#PatchFilesByFileId>